

The slide features a light gray background with several decorative elements: a large white circle in the top-left corner, a purple circle in the top-left, a purple horizontal bar in the top-right, a purple circle in the bottom-right, a large white circle in the bottom-right, and a purple horizontal bar in the bottom-left.

Prog II:

Introdução à

Orientação a Objetos

Prof^a. Tainá Isabela

Paradigma de Orientação a Objetos

A Programação Orientada a Objetos (POO) constitui um paradigma de desenvolvimento de software baseado na modelagem de componentes do mundo real.

Esse modelo de computação aproxima a estrutura do código aos elementos conceituais do negócio, facilitando a manutenção e a escalabilidade de sistemas complexos.

O foco central reside na representação de dados e comportamentos integrados em unidades funcionais independentes.

Origem e Evolução do C++

Desenvolvido por Bjarne Stroustrup como uma extensão da linguagem C, o C++ introduziu o suporte à Orientação a Objetos mantendo a alta performance e o controle de baixo nível.

A linguagem permite o desenvolvimento de sistemas robustos através da manipulação direta de memória e suporte a múltiplos paradigmas.

Atualmente, permanece como uma das ferramentas mais eficientes para softwares que exigem alto desempenho computacional.

O Conceito de Classes

Uma classe atua como um molde, modelo ou especificação abstrata que define as **características e comportamentos** que os objetos **gerados a partir** dela possuirão.

Trata-se de um tipo de dado personalizado estruturado pelo programador para representar uma entidade do sistema.

Na sintaxe do C++, a declaração de uma classe delimita o escopo de suas variáveis e funções membros através de chaves.

Definição de Objetos

O objeto consiste na instância real e concreta de uma classe previamente definida, ocupando um espaço físico na memória do computador durante a execução do programa.

Se a classe é interpretada como o projeto de engenharia, o objeto representa a construção física executada com base em tal especificação.

Múltiplos objetos independentes podem ser criados a partir de uma única estrutura de classe.

Atributos de uma Classe

Os atributos, também denominados variáveis de membro, representam o estado ou as características intrínsecas de uma classe.

Eles definem as propriedades informacionais que cada objeto armazenará individualmente ao longo de seu ciclo de vida.

No ambiente C++, os atributos podem abranger desde tipos primitivos, como inteiros e caracteres, até estruturas complexas e outras classes.

Métodos de uma Classe

Os métodos representam as funções membros associadas a uma classe, determinando as ações e comportamentos que os objetos são capazes de executar.

Através dos métodos, o objeto manipula seus próprios atributos internos ou interage com outros elementos do sistema.

Estas funções delimitam as operações permitidas sobre os dados, garantindo o controle operacional da estrutura.

Diferença entre Classe e Objeto

A diferenciação reside na natureza conceitual de cada termo: a classe atua estritamente no plano abstrato e do design do código fonte, não consumindo memória para dados.

O objeto, por sua vez, manifesta-se em tempo de execução, possuindo identidade própria e valores específicos atribuídos aos seus campos.

A classe define o potencial de comportamento, enquanto o objeto executa a ação concreta.

O Princípio do Encapsulamento

O encapsulamento consiste na técnica de desenvolvimento que visa ocultar os detalhes internos de implementação de uma classe, expondo apenas uma interface segura.

Esse princípio protege os dados internos contra acessos indevidos e modificações arbitrárias provenientes de rotinas externas ao objeto.

Trata-se de um pilar fundamental para garantir a segurança, integridade e o baixo acoplamento do software.

Modificadores de Acesso: Public

O modificador de acesso **public** estabelece que os membros associados — sejam atributos ou métodos — estarão plenamente acessíveis por qualquer outra parte do programa.

Elementos públicos compõem a interface externa de comunicação do objeto, permitindo a interação direta com o ecossistema do software.

Recomenda-se seu uso majoritariamente para métodos de **controle e inicialização**.

Modificadores de Acesso: Private

O modificador `private` restringe o acesso aos membros da classe exclusivamente para as funções pertencentes à própria estrutura interna daquela classe.

Funções externas, objetos instanciados ou outras rotinas do sistema ficam impedidos de visualizar ou alterar diretamente tais componentes.

Por convenção de segurança, os atributos de uma classe devem ser resguardados sob este nível de isolamento.

Métodos de Acesso: Getters

Os métodos de leitura, popularmente denominados getters, são funções públicas projetadas especificamente para retornar o valor de um atributo privado ao solicitante externo.

Esse mecanismo permite que o sistema consulte o estado do objeto sem possuir permissão para modificá-lo diretamente.

A implementação assegura o controle de fluxo e preserva o encapsulamento estrito dos dados.

Métodos de Modificação: Setters

Os métodos de escrita, conhecidos como setters, consistem em funções públicas desenvolvidas para alterar o valor de atributos privados de forma controlada.

A utilização deste intermediário permite a inserção de regras de validação e filtros de consistência antes de efetivar a alteração na memória.

Evita-se, com isso, a corrupção do estado interno do objeto por dados inválidos.

Método Construtor

O construtor constitui uma função membro especial executada de forma automática no momento exato em que um objeto é instanciado na memória.

Possui obrigatoriamente o mesmo nome da classe e não apresenta tipo de retorno, servindo primordialmente para inicializar os atributos primários.

O C++ permite a sobrecarga de construtores para oferecer diferentes formas de inicialização.

Método Construtor

O construtor constitui uma função membro especial executada de forma automática no momento exato em que um objeto é instanciado na memória.

Possui obrigatoriamente o mesmo nome da classe e não apresenta tipo de retorno, servindo primordialmente para inicializar os atributos primários.

O C++ permite a sobrecarga de construtores para oferecer diferentes formas de inicialização.

Sintaxe de Declaração em C++

A criação de uma classe em C++ exige o uso da palavra-chave `class`, seguida do identificador e de um bloco delimitado por chaves, finalizado obrigatoriamente por ponto e vírgula.

Dentro do bloco, os modificadores `public:` e `private:` organizam a visibilidade dos membros.

A correta estruturação sintática é avaliada pelo compilador antes da geração do código executável.

Instanciação de Objetos na Prática

No C++, a instanciação estática de um objeto assemelha-se à declaração de uma variável convencional, utilizando o nome da classe seguido pelo identificador do objeto.

O operador ponto . é empregado para invocar os métodos públicos ou acessar atributos permitidos a partir da instância criada.

Esse processo aloca automaticamente o espaço necessário na pilha de memória do sistema.

Organização do Código: Arquivos .h e .cpp

Em projetos profissionais com C++, recomenda-se a separação estrutural da classe em dois arquivos distintos para otimizar a compilação.

O arquivo de cabeçalho (.h ou .hpp) armazena estritamente a declaração da estrutura, atributos e protótipos de métodos.

O arquivo de implementação (.cpp) abriga o desenvolvimento do código lógico e escopo das funções.

Arquivos .h – Exemplo

```
#ifndef PESSOA_H
#define PESSOA_H

#include <string>

class Pessoa {
private:
    std::string nome;
    int idade;

public:
    Pessoa(std::string n, int i);    // Protótipo do construtor
    void apresentar();              // Protótipo do método
};

#endif
```

Arquivos .cpp – Exemplo

```
#include "Pessoa.h"
#include <iostream>

// Implementação do Construtor usando o escopo Pessoa::
Pessoa::
Pessoa::Pessoa(std::string n, int i) {
    nome = n;
    idade = i;
}

// Implementação do Método usando o escopo Pessoa::
void Pessoa::apresentar() {
    std::cout << "Nome: " << nome << ", Idade: " << idade << "
anos." << std::endl;
}
```

Arquivos .cpp – Exemplo (main)

```
#include "Pessoa.h" // Inclui o contrato da classe para  
que o main saiba que ela existe  
#include <iostream>
```

```
int main() {  
    // Instanciação de um objeto da classe Pessoa  
    utilizando o construtor definido  
    Pessoa pessoa1("Carlos Souza", 21);  
  
    // Chamada do método público da classe Pessoa  
    pessoa1.apresentar();  
  
    return 0;  
}
```

Organização do Código: Arquivos .h e .cpp

meu_projeto/

- Pessoa.h <- Apenas os protótipos e a estrutura da classe
- Pessoa.cpp <- A lógica dos métodos da classe (Pessoa::apresentar)
- main.cpp <- A função principal que inicia a execução e usa a classe

Organização do Código: Arquivos .h e .cpp

Quando o comando de compilação `g++ main.cpp Pessoa.cpp -o programa` é executado:

O compilador lê o `main.cpp` e vê que ele usa a classe `Pessoa`, graças ao `#include "Pessoa.h"`, o compilador sabe quais métodos e atributos a classe possui.

O compilador processa o `Pessoa.cpp` para obter o código binário das funções. O **linker** une o ponto de início (`main`) com as implementações de `Pessoa`, gerando o arquivo executável final.

Questionário



Exercícios

1. Implemente a classe Gato com atributos privados nome, idade e peso. O construtor deve inicializar os dados, e o método público setter do peso deve impedir a inserção de valores menores ou iguais a zero.
2. Crie a classe ContaBancaria contendo os atributos privados numeroConta e saldo (iniciado em 0). Forneça apenas o método getter para o saldo e controle as modificações estritamente pelos métodos públicos depositar e sacar.
3. Desenvolva a classe Termometro com o atributo privado temperaturaCelsius (iniciado em 25.0 pelo construtor) e um setter que bloqueie valores abaixo de -273.15 . Adicione os métodos públicos getTemperaturaCelsius e getTemperaturaFahrenheit, sendo que o segundo deve calcular a conversão sem usar um novo atributo.